

Introduction

In today's world of software development, building scalable and maintainable systems is key to ensuring long-term success. One of the essential concepts that play a significant role in achieving this goal is Dependency Injection. This technique allows developers to manage the relationships between various components of a system in a more flexible and efficient manner. By decoupling components from their dependencies, Dependency Injection promotes cleaner, more modular code that is easier to test and maintain.

To gain a better understanding of **Dependency Injection**, let's begin by examining the problem it aims to solve.

The Problem

Imagine we are building an **OrderService** and we write the following code:

```
class OrderService {  
  
    private InventoryService inventory = new InventoryService();  
  
    private PaymentService payment = new RazorpayPayment();  
  
    private NotificationService notification = new NotificationService();  
  
    public void checkout(Order order) {  
  
        inventory.blockItems(order);  
  
        payment.process(order);  
    }  
}
```

```
        notification.sendConfirmation(order);  
    }  
}
```

While this code looks simple and functional at first, there are several issues that arise when we try to scale it or make changes:

1. Hardcoded Logic

The **OrderService** is tightly coupled with specific implementations of the **InventoryService**, **PaymentService**, and **NotificationService**. This makes the code rigid. For example, if we want to switch from **Razorpay** to **Stripe**, we would need to manually change the code everywhere the **RazorpayPayment** class is used.

2. Difficult to Test

Since the dependencies (**InventoryService**, **PaymentService**, and **NotificationService**) are hardcoded within the **OrderService** class, it becomes extremely difficult to test the logic of **OrderService** in isolation.

For example, if we want to test the **checkout** method, we would need to hit real payment APIs, which is not ideal for unit testing.

3. Scalability Issues

As the application grows, we may want to introduce more payment providers, different inventory systems, or notification services. Each time we add a new dependency, we need to modify the **OrderService** class, which creates scalability issues in larger systems.

Let's now understand how **Dependency Injection** can help solve these issues and provide a cleaner, more maintainable design.

Understanding Dependency Injection

What are *Dependencies*?

In software design, a **dependency** refers to any object that a class needs in order to function properly. For example, in our earlier **OrderService**, the dependencies are **InventoryService**, **PaymentService**, and **NotificationService**. These services are needed for the **OrderService** to carry out its **checkout** operation.

Definition

Dependency Injection (DI) is a design pattern in which an object receives its dependencies from an external source rather than creating them itself.

In simple terms, instead of creating objects directly inside a class, you **"inject"** them from outside.

Real-life Analogy

Imagine you are a chef in a restaurant. You need several ingredients (e.g., tomatoes, onions, spices) to cook a dish. Now, instead of growing these ingredients in your garden (creating them yourself), you could simply ask the supplier to deliver them to your kitchen (injecting the dependencies). This way, you don't need to worry about the sourcing of ingredients yourself, making your job simpler and more flexible.

Similarly, in software development, **Dependency Injection** allows you to focus on what your class does (like cooking a dish), while an external entity (like a supplier) takes care of providing the necessary dependencies.

Why Dependency Injection?

- **Flexibility:** It allows you to easily swap out dependencies without changing the core logic of the class.
 - **Loose Coupling:** By injecting dependencies, your classes become decoupled from the specific implementations, making your system easier to maintain and extend.
 - **Testability:** Since dependencies are injected, it's easier to replace them with mock objects or stubs for unit testing.
-

Implementing Dependency Injection

In the previous section, we discussed the issues in the **OrderService** class. Now, let's see how we can fix these issues using **Dependency Injection (DI)**.

Here is the refactored code using Dependency Injection:

```
import java.util.*;
```

```
class OrderService2 {
```

```
    private InventoryService inventory;
```

```
    private PaymentService payment;
```

```
    private NotificationService notification;
```

```
    // Constructor Injection - Dependencies are injected through the  
    constructor
```

```
    public OrderService2(InventoryService inventory,
```

```
                        PaymentService payment,
```

```

        NotificationService notification) {

    this.inventory = inventory;

    this.payment = payment;

    this.notification = notification;
}

public void checkout(Order order) {

    inventory.blockItems(order);

    payment.process(order);

    notification.sendConfirmation(order);
}
}

// Client-side code

class Main {

    public static void main(String[] args) {

        // Injecting dependencies manually (Constructor Injection)

        OrderService2 orderService2 = new OrderService2(

            new InventoryService(),

            new RazorpayPayment(),

            new NotificationService()

```

```
);

// Now, we can use the orderService2 to perform operations
orderService2.checkout(order);
}
}
```

Let's understand how the above code fixes the earlier discussed issues.

1. Loose Coupling

In the refactored code, the **OrderService2** class no longer creates its dependencies internally. Instead, it receives the required services (**InventoryService**, **PaymentService**, and **NotificationService**) via its constructor. This decouples the class from specific implementations, which makes it more flexible.

2. Testability

Since the dependencies are injected, we can now easily provide mock implementations of these services for testing. For example, while testing, we could pass mock services instead of real ones, avoiding the need to hit actual payment gateways or databases.

3. Scalability

If we want to switch from **Razorpay** to **Stripe** (or any other payment provider), we only need to inject the new **PaymentService** **implementation** without touching the **OrderService2** class. This makes the system easier to extend and maintain.

Client-side Dependency Injection:

In the client-side code (e.g., in the **Main** class), we create an instance of **OrderService2** and inject its dependencies through the constructor. By doing this, we gain the **flexibility** to choose which implementations of the dependencies to use.

For example, we can easily swap **RazorpayPayment** with another payment service, depending on the requirements.

Advantages of Dependency Injection (DI)

Dependency Injection brings several advantages to software design. Here are the key benefits of using DI:

1. Swappable Components

Since the dependencies are injected rather than hardcoded, you can easily replace one component with another.

For instance, switching from one payment service to another (e.g., from Razorpay to Stripe) becomes straightforward without changing the core logic of the **OrderService**.

2. Testable with Mocks

With DI, you can inject mock implementations of your services when running tests. This makes unit testing easier because you don't need to call external services (e.g., payment APIs or database operations). You can replace them with mock objects that simulate their behavior.

3. Follows Dependency Inversion Principle (D in SOLID)

Dependency Injection adheres to the **Dependency Inversion Principle (DIP)**, one of the key principles of SOLID design. DIP states that high-level modules should not depend on low-level modules; both should depend on abstractions. DI enables this by allowing classes to depend on abstractions (interfaces) instead of concrete implementations.

4. Open to Extension, Closed to Modification

This is part of the **Open/Closed Principle (O in SOLID)**, which DI supports. By injecting dependencies, your code is open to extension (e.g., adding new payment types, notification systems, etc.) without modifying existing classes. You can extend the system by simply adding new implementations, without touching the core code.

Types of Dependency Injection (DI)

There are three main types of Dependency Injection. Let's dive deep into each type.

1. Constructor Injection

Constructor Injection is the most commonly used form of Dependency Injection. In this approach, dependencies are passed to the class via the constructor. This ensures that the class is always instantiated with its required dependencies, which makes it easier to manage and test.

Key Points

- **Immutable Dependencies:** Once the dependencies are injected through the constructor, they cannot be changed. This makes the object immutable, which ensures better reliability and predictability.
- **Test-Friendly:** Constructor injection makes testing easier since you can inject mock dependencies during unit testing, isolating the class under test.
- **Ensures Required Dependencies:** Since all required dependencies must be provided when the object is created, you are guaranteed that the class will always have everything it needs to function properly.

Example Code

```
// Using Constructor Injection
```



```
public class OrderService {  
  
    private final PaymentService payment;  
  
    // Constructor  
  
    public OrderService(PaymentService payment) {  
  
        this.payment = payment;  
  
    }  
  
    public void checkout(Order order) {  
  
        payment.process(order);  
  
    }  
}
```

2. Setter Injection

In Setter Injection, dependencies are passed to the class via setter methods after the object has been created. This allows for mutable dependencies, meaning you can change the dependencies of the class at runtime.

Key Points

- **Mutable Dependencies:** In Setter Injection, dependencies can be set or changed **at any time** after the object is created, which gives more

flexibility in some scenarios. This contrasts with Constructor Injection, where dependencies are only set at the time of object creation and cannot be changed later.

- **Less Strict:** Unlike constructor injection, where all dependencies are required, setter injection allows for optional dependencies.
- **Can Be Misused:** One drawback is that the dependencies may not be properly set if the setter method is not called. This can lead to situations where a class is not fully initialized.

Example Code

// Using Setter Injection

```
public class OrderService {  
  
    private PaymentService payment;  
  
  
    // Setter method to inject dependencies  
    public void setPayment(PaymentService payment) {  
        this.payment = payment;  
    }  
  
    public void checkout(Order order) {  
        payment.process(order);  
    }  
}
```

3. Interface Injection

In Interface Injection, the dependency provides an injector method that will inject the dependency into the class. This type is rarely used in practice and is typically only suitable for very specific cases.

Key Points

- **Rarely Used:** This type of DI is not as commonly used as constructor or setter injection. It requires changes to the interfaces themselves and may not always be suitable for all applications.
- **Requires Interface Changes:** Since the class must implement an interface that provides the injection method, it can require significant changes to existing interfaces.
- **Unnecessary Method Implementation:** A drawback of **Interface Injection** is that every class implementing the interface is required to implement the `injectPayment` method, even if the class does not need a payment service. This can lead to unnecessary method definitions and potential code bloat in some cases.

Example Code

```
// Interface to inject PaymentService dependency
```

```
public interface PaymentInjectable {  
  
    // Method to inject PaymentService  
  
    void injectPayment(PaymentService payment);  
  
}
```

```
// Using Interface Injection
```

```
public class OrderService implements PaymentInjectable {
```

```
private PaymentService payment;

// Inject PaymentService through the method

@Override

public void injectPayment(PaymentService payment) {

    this.payment = payment; // Set the injected payment service

}


public void checkout(Order order) {

    payment.process(order);

}

}
```

Best Practices for Implementing Dependency Injection in Java

The below code contains the best practices for implementing **Dependency Injection** in Java.

```
// — Contract: defines what the client needs, not how it is done

interface NotificationService {
```

```
void send(String message);  
}
```

// — Concrete implementation of the contract

```
class EmailNotificationService implements NotificationService {  
  
    @Override  
  
    public void send(String message) {  
  
        System.out.println("Email sent: " + message);  
  
    }  
}
```

// — Client that depends on the abstraction, not the implementation

```
class UserService {  
  
    // Dependency held as an interface, promoting loose coupling  
  
    private final NotificationService notificationService;  
  
    // Constructor Injection: forces the caller to supply the dependency  
    up-front  
  
    public UserService(NotificationService notificationService) {  
  
        this.notificationService = notificationService;  
  
    }  
}
```

```
// Business logic uses the injected service

public void register(String user) {

    System.out.println("User registered: " + user);

    notificationService.send("Welcome " + user);

}

}

// — Composition Root: the only place where “new” keywords appear

class Main {

    public static void main(String[] args) {

        // Create the concrete dependency

        NotificationService service = new EmailNotificationService();

        // Inject it into the client

        UserService userService = new UserService(service);

        // Execute business operation

        userService.register("raj");

    }

}
```

Why this approach is considered best-practice

Reason	How the code achieves it
Depends on abstractions	<code>UserService</code> holds a <code>NotificationService</code> interface, not a concrete class.
Constructor Injection ensures completeness	Objects can't be created without their required dependencies, preventing partially-initialized states.
Immutability & Thread-safety	Dependencies are final ; once injected they can't change, making the object safer to use across threads.
Single Responsibility	Only the <i>composition</i> root (<code>Main</code>) knows about concrete classes; the rest of the code focuses purely on business logic.
Easy swapping & testing	Swap <code>EmailNotificationService</code> with <code>SmsNotificationService</code> , or inject a mock in tests, without touching <code>UserService</code> .
Adheres to SOLID (DIP & OCP)	High-level modules depend on interfaces, and new notification channels can be added without modifying existing code.

When Dependency Injection is Not Needed

While **Dependency Injection (DI)** is a powerful design pattern, it is not always necessary. Here are some situations where DI may not be needed:

1. Tiny Classes with zero dependencies

If a class is small and doesn't rely on any other services or components, there is no need to introduce DI. The simplicity of such classes doesn't justify the added complexity of dependency injection.

2. Static utility classes:

Static classes that provide utility functions (like string manipulation or date handling) don't require DI because they don't have any instance dependencies. Their behavior remains the same across all calls and does not change based on external factors.

3. One-off scripts or tools

For small, isolated scripts or tools used just once, the overhead of introducing DI might not be necessary. In such cases, simplicity and direct implementation may be more efficient than using a design pattern like DI.

When to Use Dependency Injection

It is best suggested to use **Dependency Injection** when you see the following symptoms:

1. Classes use new for internal collaborators

If classes are directly creating instances of their dependencies (using **new**), this tight coupling makes them hard to test and maintain. DI helps in breaking this coupling by injecting dependencies externally.

2. Cannot mock services in tests

When your classes create their dependencies internally, it becomes difficult to replace them with mock objects during testing. DI allows you to inject mock dependencies, making unit testing much easier.

3. Adding feature branches old code

If adding new features causes existing code to break or requires you to modify many parts of the system, it could be a sign of poor separation of

concerns. DI helps by making dependencies more modular, reducing the impact of changes.

4. Too many **if** statements to switch service type

If your code is full of **if** or **switch** statements to handle different types of services or behaviors, DI can help by injecting the correct service implementation dynamically, based on the configuration or environment.

Table for Symptoms and DI Solutions

Symptom	Fix with DI
Classes use new for internal collaborators	Constructor Injection
Cannot mock services in tests	Inject dependency via interface
Adding feature breaks old code	Use abstraction and inject
Too many if / switch for service type	Inject strategy implementation

Framework Dependency Injection (DI) with Spring

Till now, we have seen how to manually implement Dependency Injection via the three methods discussed earlier. However, there is another more efficient way to implement DI, which is through frameworks like **Spring**.

With **Framework DI**, the DI process is handled automatically by the framework itself. In Spring, dependencies are injected using annotations like **@Autowired** or **@Inject**. This eliminates the need for manually wiring objects, making the code cleaner, more scalable, and easier to maintain.

The framework takes care of creating and managing dependencies, reducing boilerplate code and improving application structure.